ASSIGNMENT

NAME:R.SHASHIDEEPIKA

H.NO:2303A51871

B-13

Lab 20 – Security Testing: Identifying Vulnerabilities in AI-Generated Code

Task 1 – Input Validation Check

Task:

Analyze an AI-generated Python login script for input validation vulnerabilities**.**

Instructions:

- Prompt AI to generate a simple username-password login program.
- Review whether input sanitization and validation are implemented.
- Suggest secure improvements (e.g., using re for input validation).

Expected Output:

- A secure version of the login script with proper input validation.

Prompt:

Generate a Python login program with username and password. Show an insecure version without input validation, then provide a secure version using regex for validation and explain the code with sample outputs.

Insecure version without input validation

Insecure Code:

```python
# Generate a Python login program with username and password. Show an insecure version without input validation, then provide a secure version us
# Insecure version without input validation
class LoginSystem:
    """A simple login system without input validation."""
    def __init__(self):
        self.users = {}  # Dictionary to store username and password pairs
    def register(self, username, password):
        """Registers a new user with the given username and password."""
        self.users[username] = password
        print(f"User '{username}' registered successfully.")
    def login(self, username, password):
        """Logs in a user if the username and password match."""
        if username in self.users and self.users[username] == password:
            print(f"User '{username}' logged in successfully.")
        else:
            print("Invalid username or password.")
# Example usage of the insecure version
login_system = LoginSystem()
login_system.register("user1", "password123")
login_system.login("user1", "password123")  # Output: User 'user1' logged in successfully.
login_system.login("user1", "wrongpassword")  # Output: Invalid username or password
```

Secure Code:

```python
# Secure version with regex validation
import re
class SecureLoginSystem:
    """A secure login system with input validation."""
    def __init__(self):
        self.users = {}

    def is_valid_username(self, username):
        """Checks if the username is valid (alphanumeric and between 3-20 characters)."""
        return re.match("^[a-zA-Z0-9]{3,20}$", username) is not None

    def is_valid_password(self, password):
        """Checks if the password is valid (at least 6 characters, contains at least one uppercase letter, one lowercase letter, and one digit)."
        if len(password) < 6:
            return False
        if not re.search(r"[a-z]", password):
            return False
        if not re.search(r"[A-Z]", password):
            return False
        if not re.search(r"\d", password):
            return False
        return True
```

```python
    def register(self, username, password):
        """Registers a new user with the given username and password."""
        if not self.is_valid_username(username):
            print("Invalid username. Please use 3-20 alphanumeric characters.")
            return
        if not self.is_valid_password(password):
            print("Invalid password. Please use at least 6 characters with uppercase, lowercase, and a digit.")
            return
        self.users[username] = password
        print(f"User '{username}' registered successfully.")

    def login(self, username, password):
        """Logs in a user if the username and password match."""
        if username in self.users and self.users[username] == password:
            print(f"User '{username}' logged in successfully.")
        else:
            print("Invalid username or password.")
```

```python
# Example usage of the secure version
secure_login_system = SecureLoginSystem()
secure_login_system.register("user1", "Password123")
secure_login_system.login("user1", "Password123")  # Output: User 'user1' logged in successfully.
secure_login_system.login("user1", "wrongpassword")  # Output: Invalid username or password
secure_login_system.register("us", "pass")  # Output: Invalid username. Please use 3-20 alphanumeric characters.
secure_login_system.register("user2", "pass")  # Output: Invalid password. Please use at least 6 characters with
secure_login_system.register("user3", "Password")  # Output: Invalid password. Please use at least 6 characters w
secure_login_system.register("user4", "password123")  # Output: Invalid password. Please use at least 6 character
secure_login_system.register("user5", "PASSWORD123")  # Output: Invalid password. Please use at least 6 character
secure_login_system.register("user6", "Pass123")  # Output: User 'user6' registered successfully.
```

Output:

```
User 'user1' registered successfully.
User 'user1' logged in successfully.
Invalid username or password.
User 'user1' registered successfully.
User 'user1' logged in successfully.
Invalid username or password.
Invalid username or password.
User 'user1' registered successfully.
User 'user1' logged in successfully.
Invalid username or password.
User 'user1' logged in successfully.
Invalid username or password.
Invalid username. Please use 3-20 alphanumeric characters.
Invalid password. Please use at least 6 characters with uppercase, lowercase, and a digit.
Invalid password. Please use at least 6 characters with uppercase, lowercase, and a digit.
Invalid password. Please use at least 6 characters with uppercase, lowercase, and a digit.
Invalid password. Please use at least 6 characters with uppercase, lowercase, and a digit.
User 'user6' registered successfully.
```

Justification

Input validation is a critical aspect of secure programming because it ensures that only valid and expected data is processed by the application. In this task, the AI-generated login system lacked proper validation, making it vulnerable to incorrect or malicious inputs. By implementing validation using regular expressions, the system enforces constraints on usernames and passwords, reducing the risk of attacks and improving data integrity.

Task 2 – SQL Injection Prevention

Task:

Test an AI-generated script that performs SQL queries on a database.

Instructions:

• Ask AI to generate a Python script using SQLite/MySQL to fetch user details.

• Identify if the code is vulnerable to SQL injection (e.g.,using string concatenation in queries).

• Refactor using parameterized queries (prepared statements).

Expected Output:

• A secure database query script resistant to SQL injection.

Prompt:

#Create a Python SQLite program to fetch user details. Show insecure SQL query vulnerable to SQL injection and then fix it using parameterized queries. Include explanation and output.

Code:

```python
#Create a Python SQLite program to fetch user details. Show insecure SQL query vulnerable to SQL injection and then fix it using parameterized qu
import sqlite3
# Connect to database
conn = sqlite3.connect('users.db')
cursor = conn.cursor()

# Create table
cursor.execute('''
CREATE TABLE IF NOT EXISTS users (
    id INTEGER PRIMARY KEY,
    username TEXT NOT NULL,
    email TEXT NOT NULL
)
''')

# Insert sample data (IMPORTANT FIX)
cursor.execute("INSERT INTO users (username, email) VALUES ('admin', 'admin@gmail.com')")
cursor.execute("INSERT INTO users (username, email) VALUES ('user1', 'user1@gmail.com')")

conn.commit()
conn.close()

# Insecure Function
def fetch_user_details_insecure(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    # Vulnerable query
    query = f"SELECT * FROM users WHERE username = '{username}'"
    cursor.execute(query)
    result = cursor.fetchall()
    conn.close()
    return result
# Secure Function
def fetch_user_details_secure(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
```

```python
    # Secure query
    query = "SELECT * FROM users WHERE username = ?"
    cursor.execute(query, (username,))

    result = cursor.fetchall()
    conn.close()
    return result


# Test Input (SQL Injection)
malicious_input = "admin' OR '1'='1"

print("Insecure Query Result:")
print(fetch_user_details_insecure(malicious_input))

print("\nSecure Query Result:")
print(fetch_user_details_secure(malicious_input))
```

Output:

```
Insecure Query Result:
[(1, 'admin', 'admin@gmail.com'), (2, 'user1', 'user1@gmail.com')]

Secure Query Result:

Secure Query Result:
Secure Query Result:
[]
```

Justification

SQL Injection is one of the most common and dangerous vulnerabilities in database-driven applications. This task highlights how using string concatenation in SQL queries allows attackers to manipulate queries and gain unauthorized access to data. By replacing such queries with parameterized statements, the application ensures that user input is treated strictly as data, preventing injection attacks.

Task 3 – Cross-Site Scripting (XSS) Check

Task:

Evaluate an AI-generated HTML form with JavaScript for XSS vulnerabilities.

Instructions:

- Ask AI to generate a feedback form with JavaScript-based output.
- Test whether untrusted inputs are directly rendered without escaping.
- Implement secure measures (e.g., escaping HTML entities, using CSP).

Expected Output:

- A secure form that prevents XSS attacks.

Prompt:

#Create an HTML form with JavaScript to display user input. Show an insecure version vulnerable to XSS and then fix it using safe rendering and CSP with explanation and outputs.

Code:

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>XSS Demo</title>

    <!-- CSP for secure section -->
    <meta http-equiv="Content-Security-Policy" content="default-src 'self'; script-src 'self' 'unsafe-inline';">
</head>
<body>

    <!--  Insecure Version -->
    <h1>Insecure XSS Example</h1>
    <form id="xssForm1">
        <label>Enter something:</label>
        <input type="text" id="userInput1">
        <button type="submit">Submit</button>
    </form>
    <div id="output1"></div>

    <hr>
```

```html
    <!--  Secure Version -->
    <h1>Secure XSS Example</h1>
    <form id="xssForm2">
        <label>Enter something:</label>
        <input type="text" id="userInput2">
        <button type="submit">Submit</button>
    </form>
    <div id="output2"></div>

    <script>
        // Insecure
        document.getElementById('xssForm1').addEventListener('submit', function(event) {
            event.preventDefault();
            var userInput = document.getElementById('userInput1').value;
            document.getElementById('output1').innerHTML = "You entered: " + userInput;
        });

        //  Secure
        document.getElementById('xssForm2').addEventListener('submit', function(event) {
            event.preventDefault();
            var userInput = document.getElementById('userInput2').value;
            document.getElementById('output2').textContent = "You entered: " + userInput;
        });
    </script>

</body>
</html>
```

Output:

Insecure XSS Example

Enter something: Hello Submit

You entered: Hello

Secure XSS Example

Enter something: Hello Submit

You entered: Hello

Justification

Cross-Site Scripting (XSS) occurs when untrusted user input is directly rendered in a web page without proper sanitization. This task demonstrates how using methods like innerHTML can allow attackers to inject malicious scripts. By switching to safer alternatives such as textContent, the application prevents script execution and protects users from potential attacks.

Task 4 – Real-Time Application: Security Audit of AI-Generated Code

Scenario:

Students pick an AI-generated project snippet (e.g., login form,API integration, or file upload).

Instructions:

- Perform a security audit to detect possible vulnerabilities.
- Prompt AI to suggest secure coding practices to fix issues.
- Compare insecure vs secure versions side by side.

Expected Output:

- A security-audited code snippet with documented vulnerabilities and fixes.

Prompt:

#Create an insecure code example (login/API/upload), identify its security vulnerabilities, and then provide a secure version with fixes and explanation.

Insecure Code:

```python
# Insecure Code Example
def login(username, password):
    """A simple login function that checks username and password."""
    if username == "admin" and password == "password123":
        return "Login successful!"
    else:
        return "Login failed!"
# Security Vulnerabilities:
# 1. Hardcoded Credentials: The username and password are hardcoded, making it easy for attackers to guess or find them.
# 2. No Hashing: The password is stored in plain text, which is a major security risk if the code is exposed.
# 3. No Rate Limiting: There is no mechanism to prevent brute-force attacks, allowing attackers to try multiple username and password combinatior
# 4. No Input Validation: The function does not validate the input, which could lead to injection attacks if the input is used in other parts of
```

Secure Code:

```python
# Secure Code Example
import hashlib
import time
# A simple user database with hashed passwords
user_db = {
    "admin": hashlib.sha256("password123".encode()).hexdigest()
}
def login(username, password):
    """A simple login function that checks username and password."""
    hashed_password = hashlib.sha256(password.encode()).hexdigest()
    if username in user_db and user_db[username] == hashed_password:
        return "Login successful!"
    else:
        return "Login failed!"
# Explanation of Fixes:
# 1. Hashed Passwords: The password is now hashed using SHA-256, which means that even if the code is exposed, the actual password is not easily
# 2. User Database: The credentials are stored in a user database, which can be easily updated and managed without changing the code.
# 3. Input Validation: Although not implemented in this example, input validation can be added to ensure that the username and password meet cert
# 4. Rate Limiting: To prevent brute-force attacks, a rate-limiting mechanism can be implemented, such as locking the account after a certain num
# Example Usage
print(login("admin", "password123"))  # Output: Login successful!
print(login("admin", "wrongpassword"))  # Output: Login failed!
print(login("user", "password123"))  # Output: Login failed!
```

Output:

```
Login successful!
Login failed!
Login failed!
```

Justification

Performing a security audit on AI-generated code helps identify hidden vulnerabilities that may not be obvious during development. This task allows students to critically analyze code for issues such as weak authentication, improper validation, and insecure data handling. By applying fixes, the overall security posture of the application is improved.

Task 5 – Insecure Data Serialization Check

Task:

Evaluate AI-generated code that uses pickle for saving user data.

Instructions:

- Ask AI to generate a script using pickle.load().
- Explain why untrusted pickle data is dangerous.
- Replace with safer formats like JSON.

Expected Output:

Secure serialization code using JSON instead of pickle.

Prompt:

#Create a Python program using pickle to store user data, explain its security risks, and then replace it with a secure JSON-based solution with explanation and outputs.

Insecure Code:

```python
import pickle

# Define a simple user data class
class UserData:
    def __init__(self, username, password):
        self.username = username
        self.password = password

# Create object
user = UserData("john_doe", "securepassword123")

# Insecure: Save using pickle
with open("user_data.pkl", "wb") as file:
    pickle.dump(user, file)

print("Data saved using pickle.")

#  Insecure: Load using pickle (DANGEROUS)
with open("user_data.pkl", "rb") as file:
    loaded_user = pickle.load(file)

print("Data loaded using pickle:")
print(loaded_user.username, loaded_user.password)
```

Secure Code:

```
import json
#  Secure: Save using JSON
user_data = {
    "username": user.username,
    "password": user.password
}

with open("user_data.json", "w") as file:
    json.dump(user_data, file)

print("\nData saved using JSON.")

#  Secure: Load using JSON
with open("user_data.json", "r") as file:
    safe_data = json.load(file)

print("Data loaded using JSON:")
print(safe_data)
```

Output:

```
Data saved using pickle.
Data loaded using pickle:
john_doe securepassword123

Data saved using JSON.
Data loaded using JSON:
{'username': 'john_doe', 'password': 'securepassword123'}
```

Justification

Serialization mechanisms like pickle can pose serious security risks when used with untrusted data, as they allow arbitrary code execution during deserialization. This task highlights the dangers of using such unsafe methods and demonstrates the need for secure alternatives like JSON, which does not execute code.